INFOSYS

React Native Developer Interview Preparation Roadmap

A structured 12-week study plan

JavaScript • React • React Native • TypeScript • Redux • GraphQL • Coding • Projects • Interviews

Flexible preparation guide | Suggested pace: 3–4 hours per day, 6 days per week

How to use this roadmap

This is a flexible 12-week preparation plan for the React Native Developer role you applied for at Infosys. Since your interview is not scheduled yet, focus on understanding, implementation and revision rather than rushing. The exact interview process and topics can vary by team and project.

- Study six days per week and reserve one day for revision or rest.
- Aim for 3–4 focused hours daily; adjust the pace to your availability.
- For each topic: understand it, code a small example, explain it aloud, then answer interview questions.
- Keep a notebook of mistakes, concepts to revisit and project decisions.

12-week roadmap at a glance

Weeks	Focus	Outcome
1–2	JavaScript foundations and advanced concepts	Strong language fundamentals and 30+ coding exercises
3–4	React fundamentals and advanced React	Build reusable components and understand Hooks/state/rendering
5–7	React Native fundamentals, APIs and advanced mobile development	Build a working cross-platform app
8–9	Redux, TypeScript, GraphQL, testing and tools	Use common production technologies confidently
10	Project development and explanation	One polished project and a clear architecture walkthrough
11	DSA and problem solving	60–80 basic/intermediate problems over time
12	Technical, managerial and HR preparation	Mock interviews and readiness review

Weeks 1–2: JavaScript

Week 1 — Core JavaScript

- JavaScript overview: runtime, engines, browser vs Node.js, strict mode, execution basics.
- Variables and types: var, let, const; primitive/reference values; null, undefined, Symbol, BigInt; typeof.
- Coercion and operators: == vs ===, truthy/falsy, type conversion, optional chaining, nullish coalescing.
- Control flow: if/else, switch, loops, break and continue.
- Functions: declarations, expressions, arrow functions, callbacks, higher-order functions, default/rest parameters, IIFE, pure functions.
- Arrays and objects: creation, access, mutation, destructuring, spread/rest, Object.keys/values/entries, references.

Week 2 — Advanced JavaScript

- Scope: global, function, block and lexical scope; scope chain and temporal dead zone.
- Execution: hoisting, execution context, call stack and closures.
- this, call(), apply() and bind().
- Asynchronous JavaScript: callbacks, callback hell, promises, chaining, async/await, try/catch/finally.
- Event loop: microtasks, macrotasks, timers and promise scheduling.
- Advanced concepts: prototypes, inheritance, classes, modules, shallow/deep copy, garbage collection basics, memory leaks, debounce and throttle.
- Array methods: map, filter, reduce, find, findIndex, some, every, forEach, includes, sort and flat.

Coding target: reverse a string; palindrome; second-largest number; remove duplicates; frequency count; flatten an array; group objects by property; implement debounce/throttle; write custom map/filter/reduce; implement a promise-based delay.

Weeks 3–4: React.js

Week 3 — React fundamentals

- React, JSX, components, component composition and props.
- State, props vs state, lifting state up and parent/child communication.
- Conditional rendering, lists, keys and fragments.
- Virtual DOM, reconciliation and React Fiber at a high level.
- Hooks: useState, useEffect, useRef, useContext, useReducer, useMemo, useCallback and useLayoutEffect.
- Effect dependency arrays, cleanup functions, lifecycle patterns and Strict Mode.

Week 4 — Advanced React

- Context API, prop drilling, local vs global state and when to use a state library.
- Rendering and performance: React.memo, useMemo, useCallback, batching and avoiding unnecessary re-renders.
- Controlled/uncontrolled forms, validation, React Hook Form or Formik basics.
- Custom Hooks, error boundaries, lazy loading and code-splitting concepts.
- Understand React Router concepts for web React, while focusing on React Navigation for mobile.
- Build a counter, todo app, search/filter UI, login form, product list, pagination and reusable modal.

Weeks 5–7: React Native

Week 5 — Fundamentals

- What React Native is; React vs React Native vs Flutter; cross-platform development.
- Expo vs React Native Community CLI; project structure; Metro bundler; Android/iOS tooling.
- Core components: View, Text, Image, ImageBackground, TextInput, ScrollView, FlatList, SectionList, Pressable, Modal, ActivityIndicator, RefreshControl, KeyboardAvoidingView and StatusBar.
- Styling: StyleSheet, Flexbox, dimensions, responsive layouts, positioning, margins/padding, shadows and platform-specific styles.
- Navigation: stack, bottom tabs, drawer, nested navigation, route params, protected flows and deep linking.

Week 6 — APIs, storage and device capabilities

- REST basics and HTTP methods: GET, POST, PUT, PATCH and DELETE; status codes and headers.
- Fetch vs Axios; interceptors; loading/error states; timeouts; retries; pagination; infinite scrolling and pull-to-refresh.
- Storage choices: AsyncStorage, SecureStore, SQLite and MMKV; caching and offline data.
- Authentication: JWT, access/refresh tokens, expiry, logout, session handling and secure token storage.
- Device features: camera, gallery, location/GPS, push notifications, permissions, file handling, clipboard and network information.

Week 7 — Advanced mobile development

- AppState and foreground/background behavior; app launch and screen focus.
- Older architecture: JavaScript thread, native/UI thread, Bridge, asynchronous communication and serialization.
- New Architecture: JSI, Fabric, TurboModules and Codegen; understand the purpose and high-level data flow.
- Android vs iOS behavior, Platform API, Platform.select(), platform-specific files and native dependencies.
- Performance: FlatList virtualization, stable keys, memoization, image optimization, reducing re-renders, memory leaks, startup time and bundle size.
- Animations and background tasks at an overview level; debugging and crash investigation.

Weeks 8–9: Advanced tools and ecosystem

Week 8 — Redux, TypeScript and GraphQL

- Redux concepts: store, actions, reducers, dispatch, selectors and middleware.
- Redux Toolkit: configureStore, createSlice, createAsyncThunk, Redux DevTools and API state.

- Compare Redux, Context API and Zustand; know when global state is justified.
- TypeScript: primitive types, type inference, interfaces, type aliases, unions/intersections, enums, generics, optional/readonly properties, any vs unknown, never and utility types.
- Type React Native props, navigation parameters and API responses.
- GraphQL: schema, types, queries, mutations, subscriptions, variables, fragments and resolvers.
- Apollo Client: provider, useQuery, useMutation, useLazyQuery, caching, pagination and error handling.

Week 9 — Git, testing, debugging and delivery

- Git/GitHub: clone, add, commit, push, pull, fetch, branches, merge, rebase basics, conflicts, pull requests and code reviews.
- Testing: unit, integration and end-to-end testing; Jest; React Native Testing Library; mocks and coverage; Detox overview.
- Debugging: React Native DevTools, Metro logs, Android Logcat, Xcode logs, network inspection and crash reporting.
- Build/release: debug vs release, APK/AAB, iOS IPA basics, signing, environment variables, build variants, EAS Build and Fastlane basics.
- CI/CD and app-store deployment concepts.

Week 10: Build a project

Project 1 — E-commerce mobile application

- Screens: login/register, home, product list, product details, cart, wishlist, checkout and profile.
- Features: search, filtering, pagination, form validation, loading/error states and token-based authentication.
- Suggested stack: React Native, TypeScript, React Navigation, Redux Toolkit, Axios, REST API, secure storage and Jest.
- Be ready to explain state flow, API layer, navigation, authentication, error handling, performance and testing.

Optional Project 2 — Employee management app

- Employee login, dashboard, employee list/details, attendance, tasks, status updates, search and profile.
- Add role-based access, notifications and API integration. Use GraphQL for selected operations if practical.
- Only claim features you personally implemented and can explain.

Project explanation questions

1. Explain the application architecture and folder structure.
2. What did you personally implement?
3. Why did you choose React Native and your state-management approach?
4. How did you integrate APIs and handle failures?
5. How did you manage authentication and token security?
6. How did you optimize lists and rendering?
7. How did you test and debug the app?
8. Describe a difficult bug and how you fixed it.
9. What would you improve in a future version?

Week 11: Data structures and algorithms

You do not need to begin with competitive programming. Focus on writing correct, readable JavaScript and explaining time and space complexity.

Topic	What to practise	Priority
Arrays	Searching, sorting, duplicates, rotations, two-sum	Very high
Strings	Reverse, palindrome, anagrams, frequency, substrings	Very high
Objects/hash maps	Frequency maps, grouping, lookups	Very high

Topic	What to practise	Priority
Two pointers/sliding window	Pair sum, palindrome, subarray problems	High
Searching/sorting	Linear/binary search; basic sorting; merge sort idea	High
Stack/queue	Balanced brackets, basic operations	Medium
Recursion/linked lists/trees	Fundamentals and traversal	Medium
Complexity	Big O time and space analysis	Very high

- Target 60–80 problems over time, not in one week.
- Start with easy problems and explain your approach before coding.
- Test edge cases: empty input, one item, duplicates, negative values and large input.

Week 12: Interview readiness

Technical interview topics

- JavaScript: closures, event loop, promises, async/await, hoisting, this, copying and array methods.
- React: rendering, Hooks, dependency arrays, memoization, Context API, forms and custom Hooks.
- React Native: architecture, Bridge vs New Architecture, FlatList vs ScrollView, navigation, AppState, permissions, platform differences and performance.
- Ecosystem: Redux Toolkit, TypeScript, REST vs GraphQL, Apollo, testing, debugging and secure storage.
- Live coding: search, API fetch, pagination, form validation, debounce, custom Hook and Redux cart.

Managerial and behavioral round

- Prepare examples about project ownership, production bugs, deadlines, changing requirements, teamwork, disagreement, code reviews and communication with QA/backend teams.
- Use STAR: Situation, Task, Action, Result. Be specific about your own contribution and the outcome.
- For scenarios, explain how you clarify the issue, assess impact, communicate trade-offs, prioritize and verify the result.

HR preparation

- Tell me about yourself; why Infosys; why React Native; strengths and development areas; career goals.
- Relocation to Pune, work preferences, salary expectations and joining availability.
- If asked about an education gap, explain it truthfully and briefly, including your NIMCET preparation if relevant.

Daily and weekly study routine

Time	Activity
60 min	Learn a new concept
45 min	JavaScript or DSA coding
60 min	React/React Native implementation
30 min	Interview questions; answer aloud
30 min	Revision and error notebook
15 min	Summarize what you learned

- Monday–Thursday: learn and practise new concepts.
- Friday: implement a feature or mini-project.
- Saturday: timed coding test and mock interview.
- Sunday: light revision, catch-up or rest.

Recommended learning resources

JavaScript.info — JavaScript fundamentals and advanced concepts
https://javascript.info/

React documentation — React concepts and Hooks
https://react.dev/learn

React Native documentation — Mobile components, APIs and architecture
https://reactnative.dev/docs/getting-started

Redux Toolkit — Modern Redux state management
https://redux-toolkit.js.org/

TypeScript handbook — Types, interfaces and generics
https://www.typescriptlang.org/docs/

Apollo Client — GraphQL integration
https://www.apollographql.com/docs/react

Jest — JavaScript testing
https://jestjs.io/docs/getting-started

LeetCode — Coding practice
https://leetcode.com/

HackerRank — JavaScript and problem-solving practice
https://www.hackerrank.com/

GeeksforGeeks — DSA explanations and practice
https://www.geeksforgeeks.org/

Final readiness checklist

- I can explain JavaScript fundamentals and asynchronous execution without memorized scripts.
- I can write common array, string and object solutions in JavaScript.
- I can build React components and explain Hooks, state and rendering.
- I can build a multi-screen React Native app with navigation and API integration.
- I can explain authentication, secure storage, Redux Toolkit and TypeScript.
- I understand REST and GraphQL fundamentals.
- I can discuss performance, debugging, testing and platform differences.
- I can explain my project architecture and personal contribution.
- I have practised timed coding and completed mock technical and managerial interviews.
- I can communicate clearly, including when I do not know an answer.

This roadmap is a preparation guide, not a guarantee of interview questions or selection. Prioritize the skills named in the specific job description and any instructions shared by the recruiter. Keep your resume accurate and be prepared to discuss every skill and project you list.